

CarePlus Predict V2 — Plano Técnico e Arquitetura Alvo

Documento de fundação. Define o que vamos construir e por quê, antes de escrever código.
Pasta nova do projeto: `/home/lincoln-pereira/VS Code/careplus-predict_2` Pasta de referência (legado): `/home/lincoln-pereira/VS Code/careplus-predict` Autor da reconstrução: equipe técnica (mobile, backend, segurança, dados, ML) Data: maio/2026

0. Como ler este documento

Este é o **plano**, não o código. A ordem das seções segue a ordem em que vamos atacar a reconstrução. Cada seção tem três partes quando faz sentido: **decisão**, **justificativa** e **o que isso destrava**. No final há o **backlog priorizado** e os **comandos de auditoria** que você roda na sua máquina sobre o projeto antigo.

Uma nota honesta sobre o ambiente: o código novo será gerado e entregue como pacote para você descompactar em `careplus-predict_2`. A auditoria do projeto **antigo** (procurar `localhost`, `fallback`, `secrets`) roda no seu terminal — os comandos estão na seção 12. Eu não tenho acesso direto à sua máquina nem ao seu GitHub a partir daqui.

1. Leitura do produto atual (a partir das telas)

O que as 11 telas revelam sobre o app de hoje:

Fluxo de paciente

- Login com seletor Paciente / Médico (segmented control azul-petróleo sobre fundo escuro).
- Home: gauge de score de saúde (0-100, "95 Estável"), cards de FC média, passos, sono e progresso, seção "Análise de Risco" e "Recomendações".
- Meus Dados: estatísticas semanais de passos (média, máximo, mínimo, total) e lista de dados diários.
- Métricas: resumo com emojis, gauge de progresso semanal, ícone de smartwatch no topo (já antecipa o wearable).
- Config: toggle "Modo Escuro", notificações, "Apple HealthKit" em Integrações, ações de exportar e limpar dados.
- Sobre: missão e versão 1.0.0.

Fluxo de médico

- Login como médico.
- Dashboard Médico: visão geral (total, alto risco, com alertas, acompanhamento, inativos, score médio) e lista de "Pacientes que Requerem Atenção".

- Pacientes: lista filtrável (Todos / Alto Risco / Com Alertas / Acompanhamento) com status online/offline, score, métricas e alertas. **Aparece um "Paciente Demo · Ativo agora"** — esse é o ponto de origem do botão de simulação.
- Config do médico: informações profissionais (Dr. Carlos Mendes, CRM), notificações, funcionalidades (telemedicina, etc.).
- Sobre médico: funcionalidades clínicas.

Sinais técnicos visíveis

- Toda tela mostra "Fonte: backend" — provável herança do problema de *fallback demo* citado no handover (o app indicava quando estava usando dados reais vs. mock).
- Integração mostra "Apple HealthKit", mas o objetivo declarado é **Samsung Health**. Há uma divergência entre o que o app sugere e a direção do produto.
- Existe um "Paciente Demo" fixo — bom para a simulação, mas hoje provavelmente está hardcoded.

Decisão de UI: preservar a identidade visual atual (azul-petróleo #1F7A99-ish sobre fundo escuro, gauges, cards arredondados). O handover é explícito: **não alterar UX/UI sem autorização**. As mudanças desta fase são funcionais e estruturais, não cosméticas — exceto onde você pediu explicitamente (tema claro/escuro e botão de simulação).

2. Princípios de arquitetura

1. **Sem dependência da máquina local.** Nada de localhost, IP fixo ou caminho de máquina no código que vai pra produção. URLs de API vêm de configuração de ambiente.
 2. **Sem fallback silencioso.** Se o backend cai, o app mostra estado de erro claro — nunca finge ter dados reais. O "Fonte: backend" pode até continuar, mas como indicador honesto, não como disfarce de mock.
 3. **Simulação é um modo explícito e isolado.** O botão liga/desliga de simulação ativa uma fonte de dados sintética claramente rotulada ("SIMULAÇÃO" / "DEMO"). Nunca se confunde com dado real de paciente. Isolada por feature flag e nunca habilitada por padrão em produção.
 4. **Segurança e LGPD desde o primeiro commit**, não como camada adicionada depois.
 5. **Wearable atrás de uma interface.** O app fala com uma abstração WearableProvider. Hoje a implementação é MockWearableProvider; amanhã entra SamsungHealthProvider sem mexer no resto do app.
 6. **Observabilidade incorporada.** Logs estruturados, request id, métricas. Sem logar dado sensível de saúde.
 7. **Tipagem ponta a ponta.** TypeScript no mobile, backend e web; contratos compartilhados.
-

3. Topologia do monorepo alvo

text

```
careplus-predict_2/
  apps/
    mobile/      # React Native (Expo bare) – paciente + médico
    web/         # Next.js – dashboard administrativo/médico
  services/
    backend/     # Fastify + Prisma + PostgreSQL
    ml/          # FastAPI – inferência clínica
  packages/
    shared-types/ # contratos TS compartilhados (DTOs, enums de risco)
    config/       # eslint, tsconfig base, prettier
  infra/
    docker/       # docker-compose para dev local (postgres, etc.)
    migrations/   # versionamento de schema (via Prisma)
  docs/
    PLANO_TECNICO.md # este documento
    ARQUITETURA.md
    SEGURANCA_LGPD.md
  .env.example    # nunca .env real no git
  README.md
```

Decisão: monorepo com pastas `apps/`, `services/`, `packages/`. Justificativa: contratos de tipo compartilhados entre mobile, web e backend evitam divergência de payload — um dos problemas históricos ("inconsistência entre telas"). Não precisa de Nx/Turborepo agora; workspaces de npm bastam e reduzem complexidade.

4. Backend — Fastify + Prisma + PostgreSQL

Stack: Node.js, Fastify, TypeScript, Prisma ORM, PostgreSQL, JWT (access + refresh), RBAC, rate limiting, OpenTelemetry, fila (BullMQ + Redis para jobs de agregação/scoring).

Módulos:

text

```
services/backend/src/
  modules/
    auth/          # login, refresh, logout, RBAC, hash de senha (argon2)
    users/         # paciente, médico, perfil, consentimento LGPD
    health/        # métricas biométricas, séries temporais, agregações
    wearable/      # ingestão de dados do provider (mock agora)
    anamnesis/     # questionário inicial, scoring, baseline clínico
    ai/            # ponte para o serviço ML
    simulation/    # modo de simulação de pacientes (liga/desliga)
    audit/         # audit logs LGPD
  infra/          # prisma client, redis, otel, config de env
  shared/         # erros, schemas zod, middlewares
  server.ts
```

Decisões-chave:

- **Auth:** access token curto (15 min) + refresh token rotativo em cookie httpOnly/secure. Senhas com argon2id. Sem JWT_SECRET no código — só via env.
- **RBAC:** papéis `PATIENT`, `DOCTOR`, `ADMIN`. Médico só vê pacientes vinculados a ele. Guard por rota.
- **Séries temporais:** tabela de leituras biométricas particionada por tempo; agregações diárias/semanais materializadas por job na fila. Isso destrava o "histórico longitudinal" e o "scoring temporal" do handover.
- **Validação:** zod em toda entrada. Nada confia no cliente.
- **Rate limiting:** por IP e por usuário nas rotas de auth.

O que isso destrava: estabiliza o backend, mata a auth espalhada, e dá base para analytics e IA.

5. Modo de simulação de pacientes (botão liga/desliga)

Você pediu **nos dois lados** (mobile e web). Desenho:

- **Backend é a fonte de verdade do estado de simulação.** Existe um flag de simulação por conta de médico/ambiente, persistido e auditado. Ligar/desligar é uma chamada autenticada que só `DOCTOR`/`ADMIN` pode fazer.
- Quando **ligado**, o backend passa a servir um conjunto de pacientes sintéticos com biometria gerada por um motor determinístico-com-ruído (curvas realistas de FC, passos, sono ao longo do dia), claramente marcados com `isSimulated: true`.
- **Mobile e web** leem esse flag e mostram um banner persistente "MODO SIMULAÇÃO ATIVO" enquanto ligado, além do próprio toggle (no app, na tela Config; na web, no header do dashboard).
- Pacientes simulados **nunca** se misturam com reais nas queries: filtro explícito por `isSimulated`.

- **Nunca liga sozinho.** Default desligado. Em produção, exige permissão e fica registrado em audit log quem ligou e quando.

Justificativa: demos previsíveis e seguras, sem o "Paciente Demo" hardcoded de hoje e sem risco de dado falso vazar pra visão clínica real.

6. Tema claro/escuro (mobile)

Hoje existe um toggle "Modo Escuro" na Config, mas o app inteiro é escuro. Vamos torná-lo real:

- **ThemeProvider** com três modos: `system` (segue o SO), `light`, `dark`.
- Tokens de cor centralizados (um arquivo de tema), não cores hardcoded espalhadas nas telas.
- Preferência persistida localmente (armazenamento seguro do device).
- Paleta clara derivada da identidade atual: mesmo azul-petróleo de marca, fundos claros, contraste AA mínimo para acessibilidade (importante em app de saúde).
- O toggle da Config passa a controlar isso de verdade, com opção "Seguir sistema".

Justificativa: acessibilidade e conforto; muitos pacientes idosos preferem tema claro de alto contraste.

7. Mobile — React Native (Expo bare)

Stack: React Native, Expo bare workflow, TypeScript, Zustand (estado leve, mais simples que Redux para este escopo), React Query (cache de servidor), React Navigation, armazenamento seguro para tokens.

text

```
apps/mobile/src/
  api/           # client http, interceptors de auth/refresh
  auth/          # contexto de sessão, guards de rota
  theme/         # tokens claro/escuro, ThemeProvider
  hooks/
  services/
  modules/
    wearable/    # WearableProvider (mock -> samsung depois)
  screens/
    patient/     # home, meus dados, métricas, config, sobre
    doctor/      # dashboard, pacientes, config, sobre
    onboarding/  # anamnese inicial
  components/    # gauge, cards, banners (ex: banner de simulação)
  store/
  utils/
```

Decisões:

- **Expo bare** (não managed) porque o Samsung Health SDK exige módulo nativo Android — managed não suporta. Confirma a direção do handover.
- **Token em armazenamento seguro**, nunca em AsyncStorage plano. Refresh automático via interceptor.
- **Sem URL hardcoded:** base URL vem de config de ambiente (`app.config.ts` + variáveis de build).

8. Wearable — abstração agora, Samsung depois

Conforme sua escolha (mock agora, SDK real depois):

```
text

WearableProvider (interface)
  └─ MockWearableProvider      # fase atual: gera dados realistas
  └─ SamsungHealthProvider     # fase futura: módulo nativo Android + Samsung Health
```

Fluxo alvo (inalterado do handover, mas atrás da interface):

```
text

Samsung Watch → Samsung Health → Native Android Module → RN Bridge → Backend → IA
```

Decisão: o app e o backend só conhecem a interface `WearableProvider` e o formato normalizado de leitura (FC, passos, sono, atividade, calorias). Trocar mock por Samsung é trocar a

implementação, não reescrever telas. **O que isso destrava:** desenvolvimento e demo hoje, sem esperar device físico nem conta de parceiro Samsung.

9. Anamnese inicial

Fluxo obrigatório antes do paciente acessar o app pela primeira vez.

```
text

services/backend/src/modules/anamnesis/
  questions/      # banco de perguntas (hábitos, histórico, comorbidades, medicação,
  scoring/         # cálculo de baseline clínico
  risk-engine/     # score de risco inicial
```

- Questionário versionado (perguntas mudam com o tempo; respostas guardam a versão).
 - Gera **baseline clínico** que personaliza o scoring e alimenta a IA.
 - Tela de onboarding no mobile; só libera o app principal após concluir.
 - Dados sensíveis ⇒ consentimento LGPD explícito antes de coletar.
-

10. IA clínica — FastAPI

Stack: FastAPI, integração com API de modelo, embeddings, pipeline de ML; RAG fica para fase futura.

```
text

services/ml/app/
  inference/       # análise preditiva, scoring
  pipelines/       # feature engineering a partir das séries temporais
  models/
  training/        # fase futura
```

- O backend chama o serviço ML; o app **não** fala direto com o ML.
 - Chave de API do modelo **só via env**, nunca no código nem no git.
 - Saída da IA: insights, sumarização clínica, tendência de risco — sempre com aviso de que é apoio à decisão, não diagnóstico.
-

11. Dashboard web — Next.js

Stack: Next.js, TypeScript, Tailwind, shadcn/ui, React Query, Recharts.

text

```
apps/web/src/  
  app/           # rotas (App Router)  
  dashboard/     # visão médica, gestão de pacientes  
  components/    # cards, gráficos clínicos, banner de simulação  
  services/      # client de API (mesmo contrato do mobile)  
  hooks/
```

- Reaproveita os contratos de `packages/shared-types`.
- Aqui também vive o toggle de simulação (lado médico/admin).
- Gráficos clínicos com Recharts; alertas e analytics populacional.

12. Segurança e LGPD

Obrigatório, desde o início.

- JWT access + refresh com rotação; argon2id para senhas.
- HTTPS obrigatório; cookies `Secure`/`httpOnly`/`SameSite`.
- **Consentimento explícito** registrado com timestamp e versão de termo, antes de qualquer coleta de saúde.
- **Audit logs** de acesso a dado clínico: quem viu o quê e quando.
- **Secrets só via env / secret manager**. Nunca no git. `.env.example` documenta as chaves sem valores.
- RBAC estrito: médico só acessa seus pacientes.
- Armazenamento seguro de token no device.
- Logs **nunca** contêm dado sensível (sem FC/diagnóstico/CPF em log).
- Direitos do titular (LGPD): exportar dados (já existe "Exportar Dados" na tela) e excluir conta/dados.

Nunca permitir: IP hardcoded, secret no git, fallback silencioso, log sensível. (Reforço direto do handover.)

13. Dados e observabilidade

- PostgreSQL com modelagem para séries temporais biométricas.
 - Agregações (diária/semanal) por job na fila, não no request.
 - OpenTelemetry no backend: traces + métricas, com request id propagado.
 - Data lake e feature store ficam para fase futura — anotado, não construído agora.
-

14. Backlog priorizado

Ordem sugerida de execução. Cada item vira uma entrega de código (zip) quando chegarmos lá.

1. **Esqueleto do monorepo** + `.env.example` + **docker-compose (Postgres/Redis)**. Base de tudo.
 2. **Backend: auth** + RBAC + **users** + **consentimento LGPD**. Mata auth espalhada.
 3. **Backend: health** (séries temporais) + **agregações**. Base de dados clínicos.
 4. **Backend: módulo de simulação** (liga/desliga, pacientes sintéticos).
 5. **Mobile: tema claro/escuro real** + **ThemeProvider** + **tokens**.
 6. **Mobile: client de API** sem URL hardcoded + **auth/refresh** + **telas de paciente**.
 7. **Mobile: WearableProvider** (mock) + **ingestão**.
 8. **Anamnese: backend** + **tela de onboarding**.
 9. **Mobile: telas de médico** + **banner/controle de simulação**.
 10. **Web: dashboard médico** + **toggle de simulação** + **gráficos**.
 11. **ML: FastAPI de inferência** + **ponte do backend**.
 12. **Observabilidade** + **audit logs** + **hardening final**.
-

15. Auditoria do projeto ANTIGO (rode na sua máquina)

Estes comandos rodam **no seu Ubuntu**, sobre a pasta de referência. Servem para extrair o que aproveitar e mapear os resíduos dos problemas históricos antes de migrar.

```
bash
```

```
# Ir para o projeto antigo
cd "/home/lincoln-pereira/VS Code/careplus-predict"

# 1) Inventário de arquivos e tamanho (ignorando node_modules)
find . -type f -not -path "*/node_modules/*" -not -path "*/.git/*" | sort > /tmp/careplus_inventory.txt
wc -l /tmp/careplus_inventory.txt
echo "--- estrutura de topo ---"
ls -la

# 2) Procurar localhost / IP hardcoded (problema histórico)
grep -RInE "localhost|127\.0\.0\.1|192\.168|10\.0\.2\.2" . \
  --include="*.ts" --include="*.tsx" --include="*.js" --include="*.jsx" \
  --include="*.json" --include="*.env*" --include="*.py" \
  --exclude-dir=node_modules --exclude-dir=.git || echo "nenhuma ocorrencia"

# 3) Procurar fallback / mock / demo (o "Fonte: backend" e o Paciente Demo)
grep -RInE "fallback|mock|demo|hardcode" . \
  --include="*.ts" --include="*.tsx" --include="*.js" --include="*.jsx" --include="*.py" \
  --include="*.json" --include="*.env*" \
  --exclude-dir=node_modules --exclude-dir=.git || echo "nenhuma ocorrencia"

# 4) Procurar secrets expostos (CRÍTICO — não deve haver nenhum no git)
grep -RInE "OPENAI_API_KEY|JWT_SECRET|SECRET|API_KEY|PRIVATE_KEY|password\s*=" . \
  --include="*.ts" --include="*.tsx" --include="*.js" --include="*.py" \
  --include="*.json" --include="*.env*" \
  --exclude-dir=node_modules --exclude-dir=.git || echo "nenhuma ocorrencia"

# 5) Conferir se algum .env está versionado no git (não deveria estar)
git ls-files | grep -E "\.env" && echo "ATENCAO: .env versionado!" || echo "ok, nenhum"

# 6) Mapear referencias a Apple HealthKit vs Samsung (divergencia das telas)
grep -RInE "HealthKit|Apple|Samsung|SamsungHealth" . \
  --include="*.ts" --include="*.tsx" --include="*.js" --include="*.java" --include="*.py" \
  --exclude-dir=node_modules --exclude-dir=.git || echo "nenhuma ocorrencia"

# 7) Salvar package.json e requirements para a gente comparar dependencias
find . -name "package.json" -not -path "*/node_modules/*" -exec echo "=== {} ===" \;
find . -name "requirements*.txt" -not -path "*/node_modules/*" -exec echo "=== {} ===" \;
echo "Salvo em /tmp/careplus_pkgs.txt"
```

Cole de volta aqui (no chat) a saída desses comandos — especialmente o conteúdo de `package.json`, `requirements.txt`, e qualquer ocorrência dos itens 2, 3, 4 e 6. Com isso eu mapeio o que reaproveitar do legado e começo a gerar o esqueleto do `careplus-predict_2` do item 1 do backlog.

16. Próximo passo concreto

Quando você quiser destravar a construção:

1. Rode os comandos da seção 15 e me cole a saída relevante.
2. Eu gero o **item 1 do backlog** (esqueleto do monorepo + docker-compose + `.env.example`) como zip pronto para descompactar em `careplus-predict_2`.
3. Seguimos backlog abaixo, uma entrega por vez, validável.

Lembrete sobre ferramentas: se a intenção é eu navegar e editar arquivos direto na sua máquina, o **Claude Code** (roda no seu terminal) é o caminho. Aqui no chat eu funciono como arquiteto e gerador de código empacotado.